SuperChat — Phase 1 Technical Report

Scope. One web chat handles one-way flight discovery and reservations, product search and cart operations, and film discovery and ticket reservations. The five organiser-supplied JSON files are the only source of catalogue records, prices, stock, seats and schedules. Reservations and carts are simulated; no payment or Phase 2 provider API is used.

Architecture and routing

`python -m superchat` starts a small Python standard-library HTTP server and serves one HTML/CSS/JavaScript chat. Each text turn passes through a language-model interpreter (`semantic.py`). The model returns a strict structured plan containing a route, one or more domain operations, filters, comparisons, references, language and explicit preferences. It never supplies catalogue facts. The backend validates the plan before execution: allowed fields and operators, evidence from the user's message, quantity and numeric coverage, and unsupported constraints. It then applies deterministic filters to repository records (`query.py`) and formats verified results. A request involving multiple domains runs each relevant operation. An unrelated request or small talk leaves domain state untouched.

The interface follows a light chat layout with a compact sidebar and cards for returned flights, products, films and showtimes. `presentation.py` builds every card from executed repository IDs; it never receives model-generated prices or availability. The official data has no product image URLs, so cards use neutral icons. Card buttons call a validated structured action endpoint for details, cart additions or showtimes. The original text answer remains available in each result

group.

This hybrid approach handles natural phrasing and numeric comparisons more flexibly than the earlier rule-only router. Its trade-off is hosted-model latency and occasional interpretation error. A rejected plan gets one bounded model repair attempt, and an unresolved or ambiguous request produces a clarification instead of an unrestricted list. `SUPERCHAT_DEBUG=1` exposes the original plan, any repair, interpreted operations and returned record IDs without exposing the API key. The earlier rule-only router remains in source for comparison; the live app uses the structured interpreter.

State and data access

A cookie identifies an in-memory session holding full history, language, explicit preferences, active domain and pending clarification. Flights, Shopping and Cinema each keep independent criteria, result IDs and selection. This preserves an earlier shortlist across topic switches. The cart belongs to the session. A shared in-memory reservation ledger deducts available seats atomically. The official JSON files are never modified.

When revisiting a saved shortlist, the interpreter may repeat old filters. The conversation layer removes only conditions that exactly match saved criteria and cite an earlier message; it still validates new conditions against the current turn. This prevents a previous search from being rejected merely because its original evidence is no longer in the current message.

Five small repository contracts expose `get` and `list_all`; `JsonRepository` is the Phase 1 adapter. Startup validates required fields, unique IDs and cross-file showtime references. Repositories return detached records, so an API adapter can replace JSON later without changing routing or domain services. Missing or invalid source files fail startup with a readable error; empty searches report no match.

Bonus features

The optional microphone control uses the browser's SpeechRecognition interface. Once enabled, the browser detects utterance boundaries and sends final transcripts through the normal chat route without holding a button. A voice-language selector supports English, Azerbaijani and Russian where the browser offers recognition.

Product photo search sends a user-selected JPEG, PNG or WebP image to the configured vision-capable model. The model proposes catalogue IDs and an estimated confidence. The backend verifies IDs and category consistency against `products.json`, presents only those records, and asks for a description when confidence is low. Photos are not saved in session history.

The interpreter selects the reply language per turn, including after a language switch. Catalogue names already exist in three languages; a separate presentation-only translation map covers descriptions, synopses and labels. Canonical value mapping and simple inflection normalization let Azerbaijani and Russian requests filter the official English-valued fields. The five official files remain unchanged.

Testing and limitations

Unit and HTTP tests cover validation, data errors, filters, routing, mixed requests, follow-ups, state preservation, cart and seat limits, cards and card actions, photo confidence and secret-file access. Live interpreters are evaluated separately because scripted unit plans do not measure model accuracy. The configured model scored 13/13 independent cases, 9/9 multi-turn cases and 12/12 multilingual cases in the final recorded runs. Gemini 2.5 Flash was faster in a probe but scored 10/13 on the independent set, so the better-tested model remains the default. See `routing_test_log.md` for case details and error analysis, and `demo_flow.md` for the five-minute demonstration.

The running process must have an OpenRouter or OpenAI API key and internet access for text and photo interpretation. Speech recognition

support and quality depend on the browser; Azerbaijani speech recognition is not guaranteed on every device, and browsers may send audio to their own online recognition service. Photo confidence is a model estimate, not a calibrated probability. Sessions and mock reservations reset on server restart and should run in one process for the demo. Only one-way flights are implemented.

Libraries and services. Runtime Python uses the Python standard library. Structured interpretation and optional vision use OpenRouter's Responses API and Chat Completions API with the `openai/gpt-5-mini` model; a direct OpenAI key is also supported. The browser microphone uses the Web Speech API. No additional runtime framework is required.